ილიას სახელმწიფო უნივერსიტეტი

კომპიუტერული გრაფიკის ალგორითმები

გიორგი ფარცხალაძე

ლექცია 8:

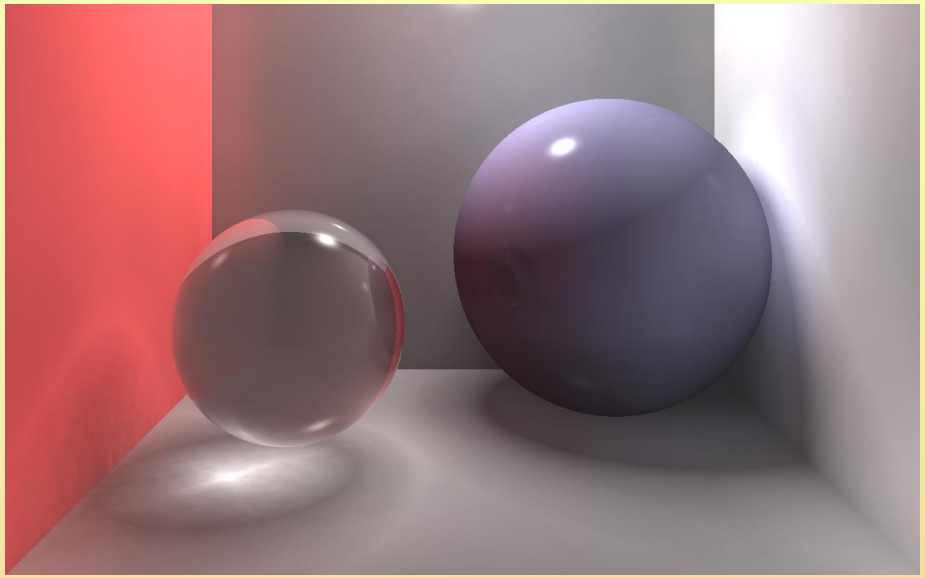
გლობალური განათება, Ray Tracing

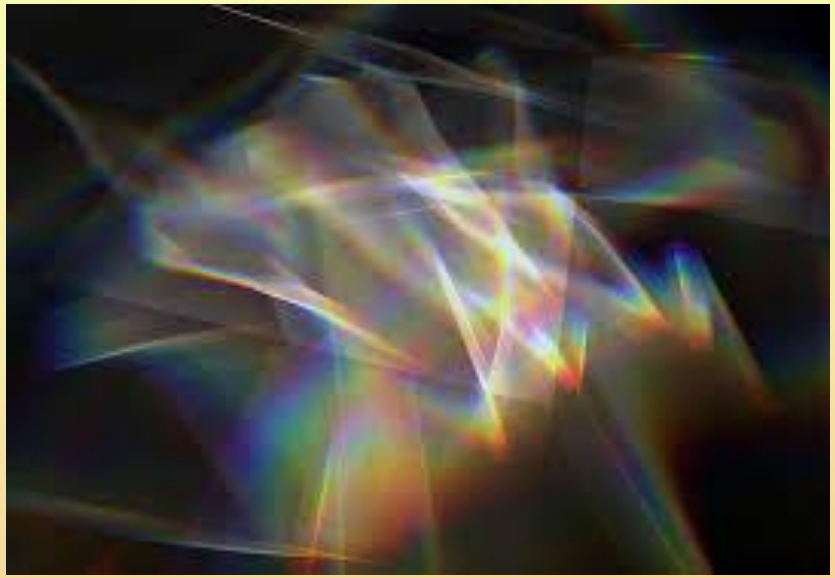
წინა ლექციაშიგანხილული ვიზუალიზაციის მეთოდები (flat, Gouraud, Phong) სწრაფია, და გამოიყენება როგორც წესი preview ვიზუალიზაციებისთვის, ანუ როცა საჭიროა სცენის ზოგადი ხასიათის ჩვენება. მაგრამ ეს მეთოდები არ არის საკმარისი ხარისხიანი გამოსახულების მისარებად.

ხარისხიანი (მითუმეტეს, ე.წ. ფოტორეალისტური) გამოსახულება ითხოვს გაცილებით მეტი ინფორმაციის დამუშავებას:

- გლობალური განათება - ობიექტების მიერ ერთმანეთის დაჩრდილვა
- არეკლვადი ზედაპირები და შუქის გარდატეხვა

- რთული ოპტიკური ეფექტები (დისპერსია, ფოტონების სიმულირება და სხვა)





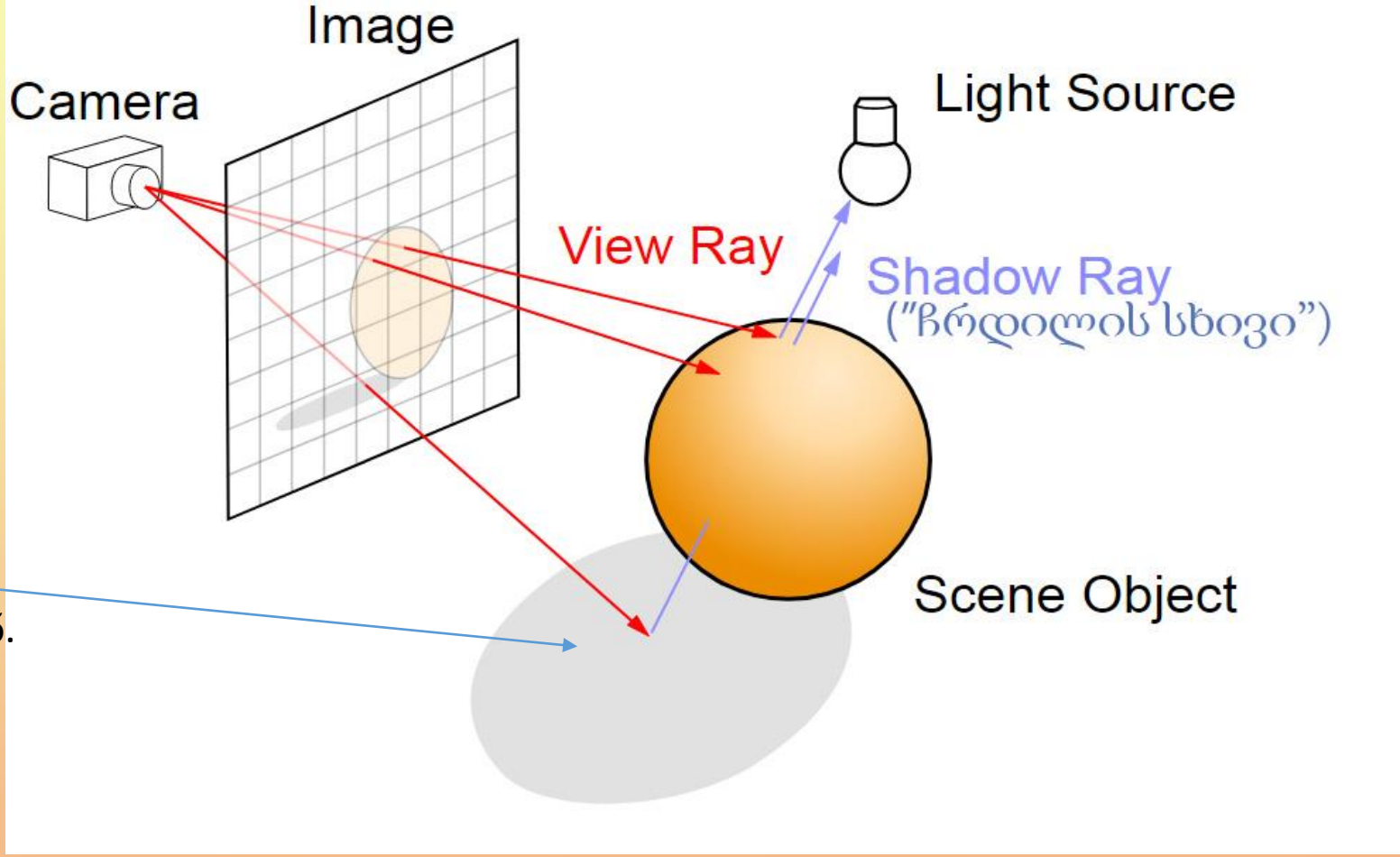
- გარემოს გათვალისწინება და მოცულობითი განათება (volume lighting)

RAY TRACING (სხივების მიდევნება)

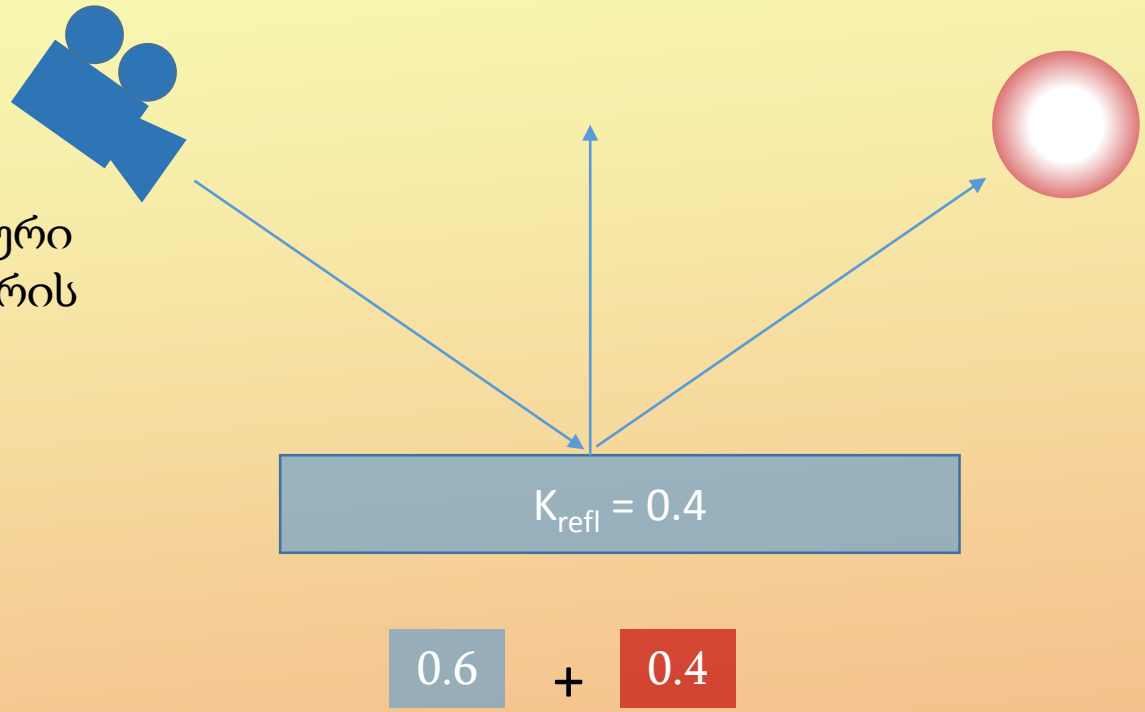
ისევე, როგორც Z-buffering-ის შემთხვევაში, ხდება სხივების გაანგარიშება ვირტუალური კამერიდან სცენისკენ, დასარენდერებელი ვირტუალური ზედაპირის „პიქსელების გავლით“.
ოღონდ ობიექტზე მოხვედრისას ვანგარიშობთ მთელ რიგ პარამეტრს:

1. ჩანს თუ არა ობიექტზე მოხვედრილი წერტილიდან ყველა შუქისწყარო?

ამისთვის მოხვედრის წერტილიდან ვავლებთ სხივებს განათების წყაროებთან. თუ ეს სხივი კვეთს სხვა ობიექტებს, ესეიგი ამ წერტილში ეს შუქის წყარო არ ჩანს, და მისგან განათებულობას არ ვანგარიშობთ. ასეთ სხივებს ეძახიან „Shadow rays“



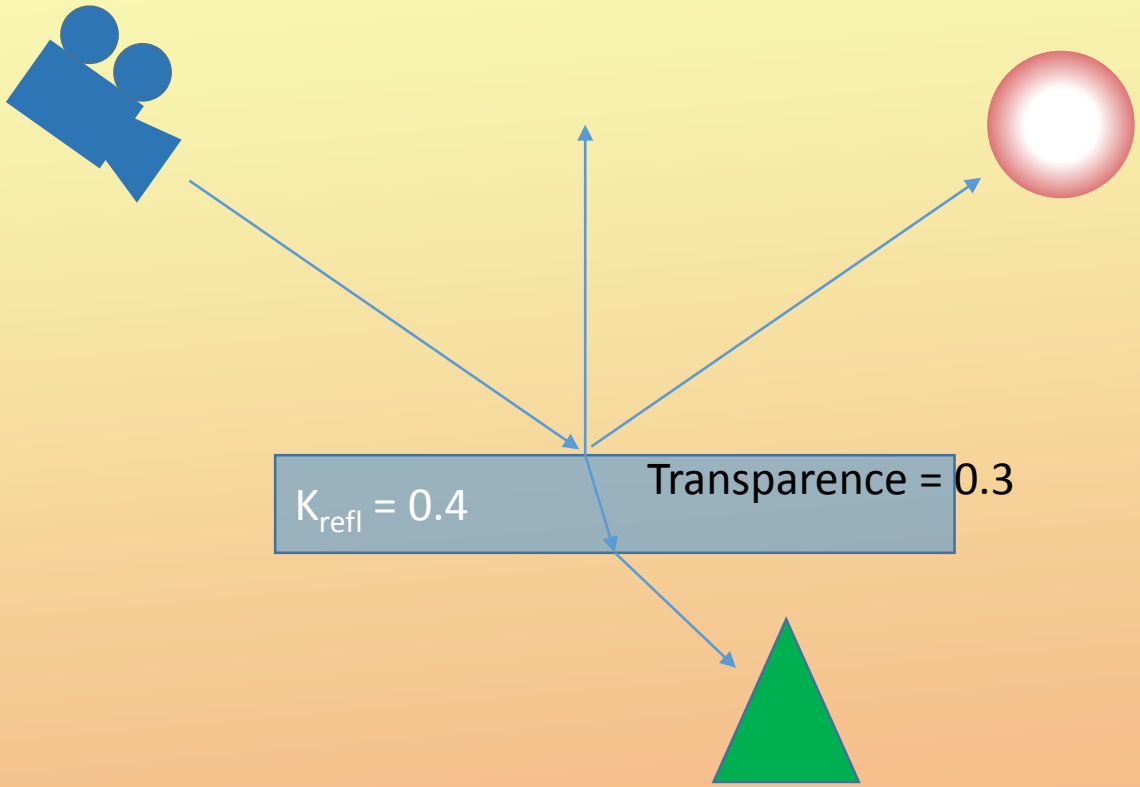
თუ ზედაპირი ირეკლავს (reflection), მაშინ გენერირდება ახალი სხივი, ამ ზედაპირიდან არეკლილი.



0.6 + 0.4

თუ ზედაპირის არეკლვადობის Krefl = 0.4, მაშინ ჯამური ფერი იქნება: 0.6-ჯერ (1-K_{refl}) გამრავლებული ზედაპირის ფერი ამ წერტილში + 0,4-ჯერ - ამ წერტილიდან დანახული სხვა ობიექტის ფერი

თუ ზედაპირი გამჭვირვალეა და ობიექტს გააჩნია სინათლის გარდატეხის კოეფიციენტი, მაშინ გენერირდება არა მარტო არეკლილი სხივი, მაგრამ ასევე გარდატეხილი სხივი.



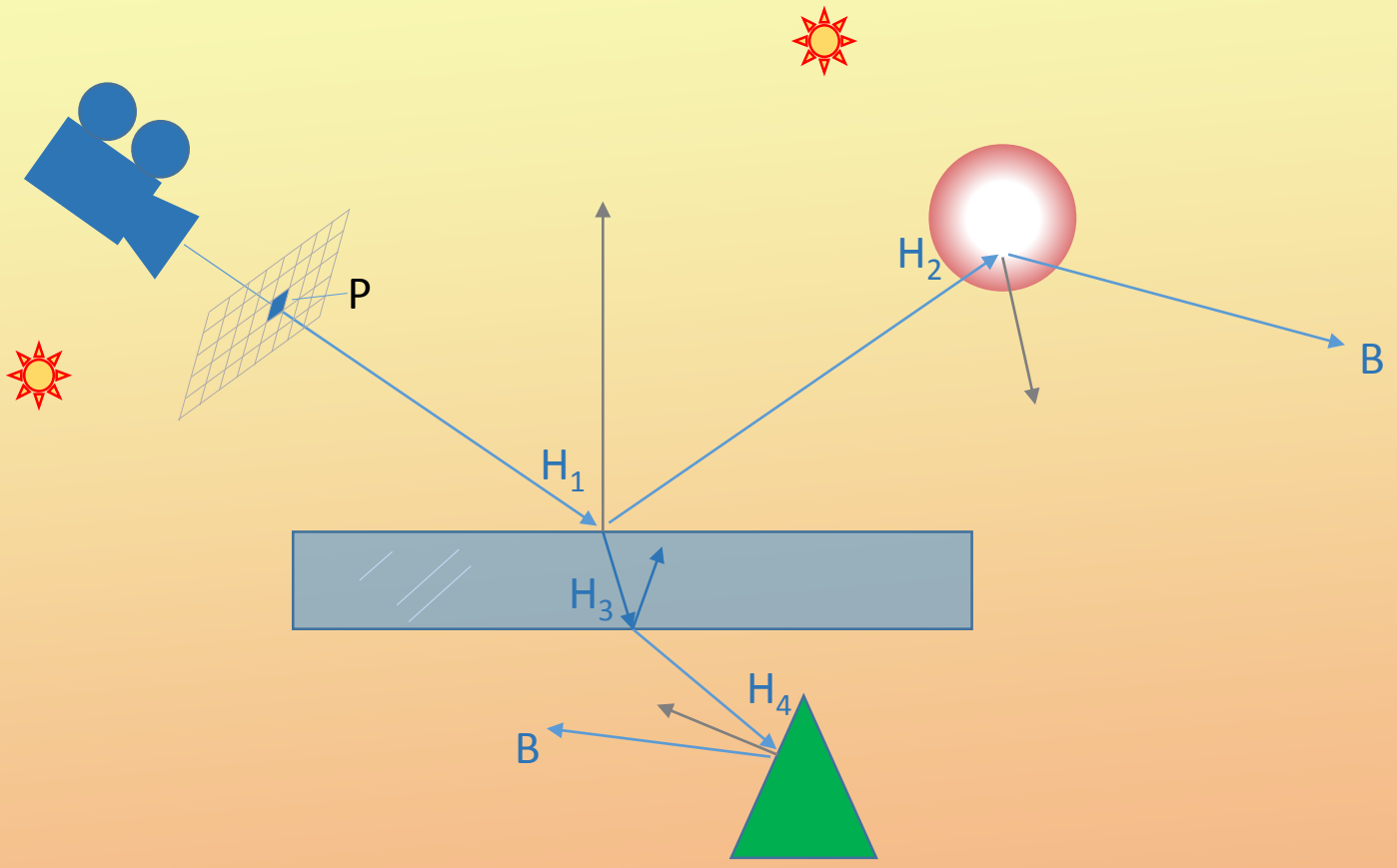
(1-0.3)*(0.6 + 0.4) + 0,3 *

როცა საჭიროა სცენაში რამდენიმე არეკლვადი და შუქის გარდამტეხი ობიექტის ჩასმა, უნდა გავითვალისწინოთ მათი მრავალჯერადი ურთიერთარეკლვა, ერთის მეორის უკან დანახვადა ა.შ.

ამ შემთხვევაში ერთ პიქსელში შეიძლება „მოვიდეს“ ინფორმაცია ბევრი სხვადასხვა ობიექტისგან

ამ შემთხვევაში ერთ პიქსელში (P) შეიძლება „მოვიდეს“ ინფორმაცია ბევრი სხვადასხვა ობიექტისგან:

H1 – წერტილი, რომელსაც ხვდება P-ში გასული სხივი.
იმის გამო, რომ ეს წერტილი ეკუთვნის ობიექტს, რომლის მასალა არის გამჭვირვალე და ირეკლავს და ასევე გარდატეხავს სხივებს:
1. ვიღებთ ამ სხეულის ფერს (სხეულის გამჭვირვალეობის Kt_1 გამო ის იქნება გამრავლებული $(1-Kt_1)$-ზე).
2. H1-დან ვანგარიშობთ არეკლილ სხივს, რომელიც მოხვდება წითელ სხეულს წერტილ H_2-ში. თუ ეს სხეულიც ირეკლავს, მაშინ ვანგარიშობთ მესამე სხივს, რომელიც სხვა ობიექტებს ვერ ხედავს და გვიბრუნებს ფონის ფერ B-ს (Background)
შესაბამისად, H2-ში დანახული ფერი იქნება წითელი გამრავლებული $(1-Kt_2)$-ზე + ფონის ფერი B, გამრავლებული Kt_2-ზე
3. H1-დან ვანგარიშობთ გარდატეხილ სხივს...
და ა.შ.



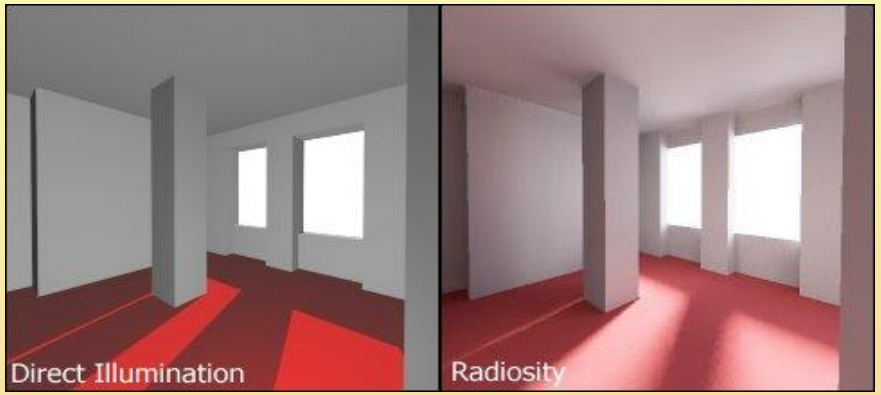
იმის გამო, რომ ერთი წერტილიდან გვიწევს რამდენიმე სხივის „გაგზავნა“, ხოლო ამ სხივების „დამიშავების“ პროცედურა ერთი და იგივეა, ray-tracing-ის პროცედურა რეკურსიულია:

```
color ray_trace ( vector_3d from, vector_3d to, int level )
        {
                color areklili = 0;
                color gardatexili = 0;
                color tavisi = 0;
                if ( level == 0) return Black;
                triangle zedapiri = nearest_triangle( from, to );
                if (zedapiri == 0) return BgColor;
                vector_3d point = ray_triangle_intersection ( from,  to, zedapiri );

                tavisi = lighting_lambert(point, from, to,  zedapiri);
                if ( zedapiri.Krf > 0)      // reflection:
                        {
                                vector_3d sait = ray_triangle_reflection ( from,  to, zedapiri );
                                areklili = ray_trace ( point, sait , level-1 );
                        }
                if ( zedapiri.Kr > 0)       // refraction:
                        {
                                vector_3d sait = ray_triangle_refraction ( from,  to, zedapiri );
                                gardatexili = ray_trace ( point, sait , level-1 );
                        }
                return tavisi*((1-zedapiri.Krf)+(1-zedapiri.Kr)) + areklili*zedapiri.Krf + gardatexili*zedapiri.Kr;
        }
```

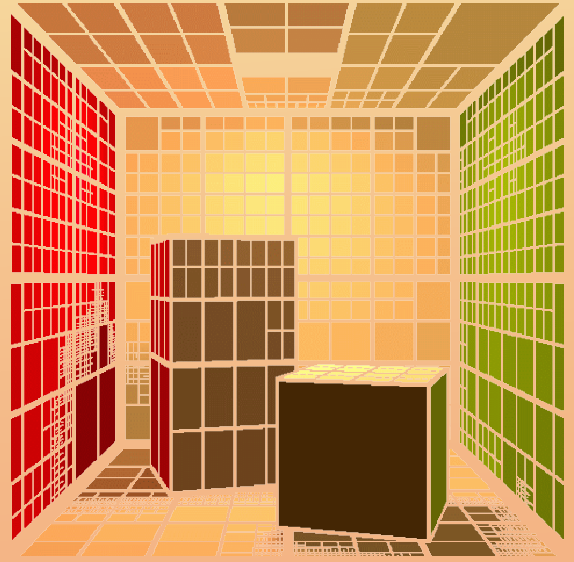
RADIOSITY

შუქის დიფუზური განაწილების მოდელირება კარგად ხდება Radiosity-ს მეთოდის მეშვეობით.



შუქი გამოდის შუქის წყაროდან (Radiosity-ს შემთხვევაში ეს ყოველთვის სასრული ზომების მქონე ობიექტია) და შემდეგ დიფუზურად ირეკლება სცენის ზედაპირებზე. არეკლილი შუქი ასევე ანათებს სცენას.

ამ სქემის დასათვლელად ხდება სცენის ობიექტების დაყოფა მცირე ელემენტებად. ყოველი ელემენტისთვის გამოითვლება ის შუქი, რომელის ამ ელემენტიდან გადის. ეს არის ჯამი - 1. ამ ელემენტის მიერ აქტიურად გამოსხივებული ენერგიის და 2. ამ ელემენტიდან არეკლილი სხვა ელემენტების შუქის.



RADIOSITY-ს განტოლება:

$$B_i = E_i + R_i \sum_{j=1}^{n} B_j F_{ij}$$

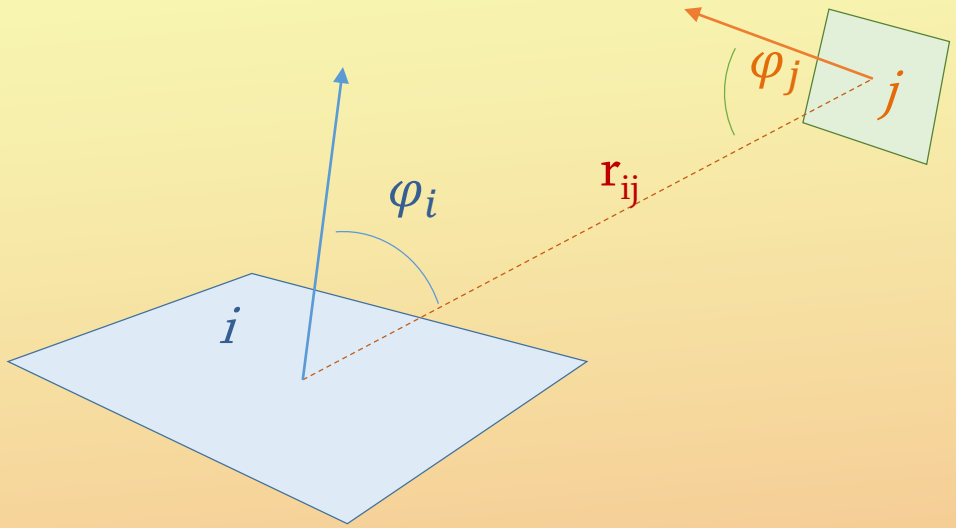
სადაც:

Bi - *i* -რი ელემენტის გამოსხივების მთლიანი ენერგიაა,
Ei (Emission) - ამ ელემენტის „საკუთარი გამოსხივებაა“,
Ri (Reflection)- ელემენტის არეკლვის კოეფიციენტი,
n - სცენაში ჩართული ელემენტების რაოდენობა,

Fij (form factor), - ფორმის კოეფიციენტი (ფაქტიურად - ხილვადობა) *i-ური და j-ური ელემენტების შორის.*

Fij (form factor), - ფორმის კოეფიციენტი (ფაქტიურად - ხილვადობა) *i-ური და j-ური ელემენტების შორის.*

$$F_{ij} = \delta_{ij} \frac{\cos\varphi_i \cos\varphi_j}{r_{ij}^2}$$



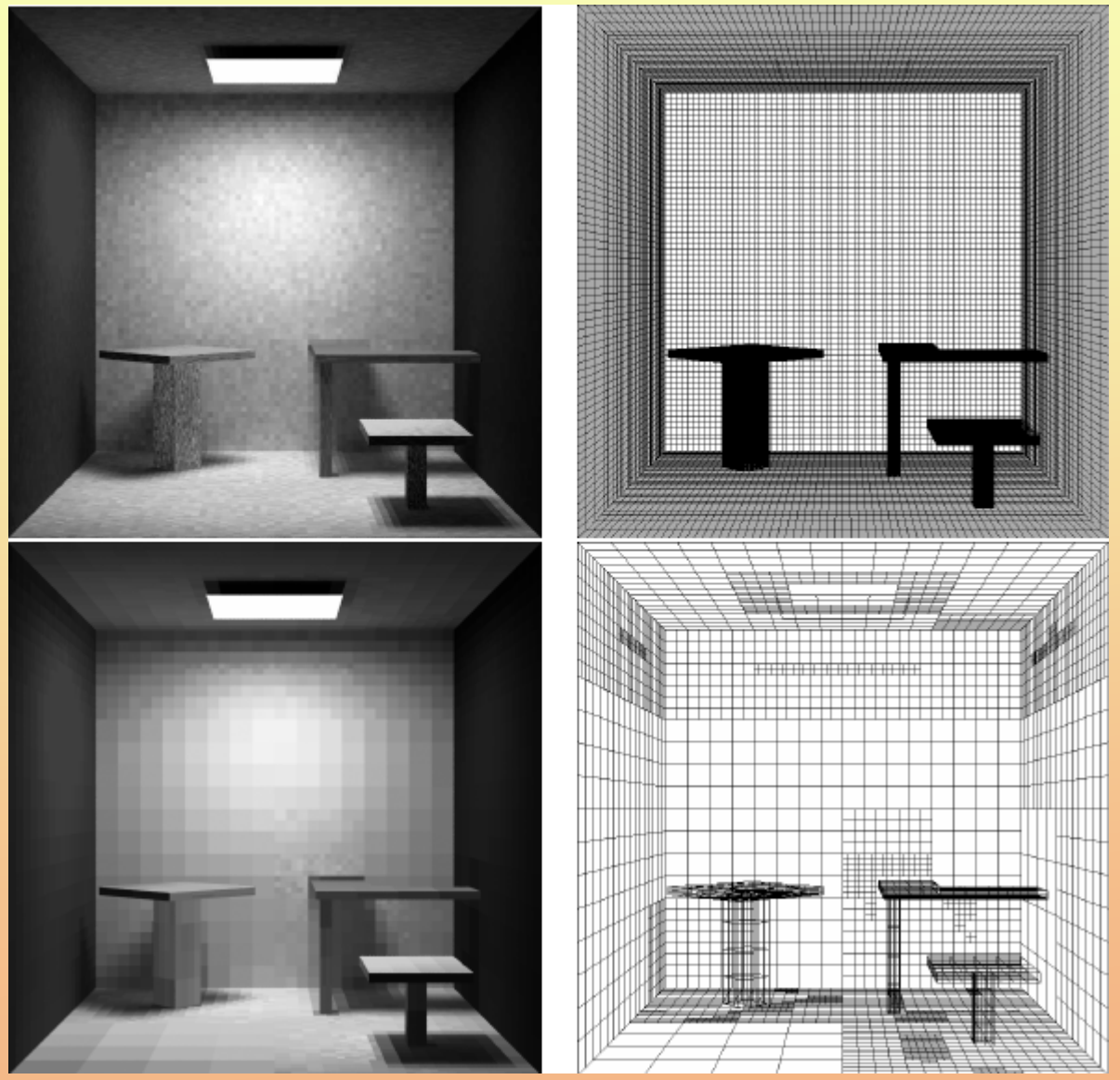
სადაც δ_{ij} – i და j ზედაპირებს შორის ხილვადობის კოეფიციენტია (თუ მათ შორის სხვა არაგამჭვირვალე სხეულია, მაშინ ეს კოეფიციენტი 0-ს ტოლია), თუ ნაწილობრივ გამჭვირვალე - მაშინ $0 < \delta_{ij} < 1$

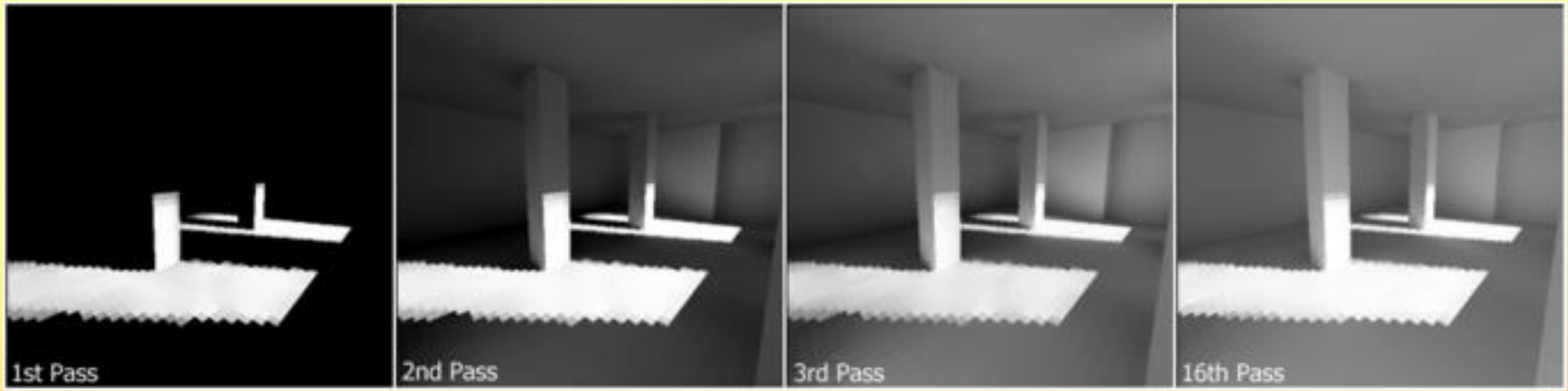
φ_i და φ_j -ზედაპირების ნორმალების კუთხეებია ზედაპირებს შორის დამაკავშირებელი ზახის მიმართ (ანუ „რამდენად უყურებენ ზედაპირები ერთმანეთს“)

r_{ij} - ელემენტებს შორის მანძილია

მუდმივი და ადაპტირებული radiosity.

იქ სადაც მეზობელ ელემენტებს შორის დიდი განსხვავებაა განათებულობაში, ელემენტები იყოფა და მათთვის თავიდან ითვლება radiosity-ს განტოლება





- მეთოდის უპირატესობები:

  1. განათების რეალისტური განაწილება
  2. განტოლების ყველა ელემენტი სტატიკურია (არ არის დამოკიდებული კამერის განლაგებაზე) და მათი გამოთვლა საჭიროა მხოლოდ ერთხელ.

- მეთოდის ნაკლები:

  1. ვერ ითვალისწინებს სარკისებულ (არადიფუზურ) არეკლვას
  2. ხშირად საჭიროებს დამატებით გამოთვლებს (remeshing)